

Full Stack Development with MERN

Project Documentation

1. Introduction

- . **Project Title:** Customer Registry — Support Complaint Management System
- . **Team Members:** K. Sai Tarun — Full Stack Developer (Frontend, Backend, Database, Deployment)

2. Project Overview

- . **Purpose:** Customer Registry centralizes customer support complaints into a single system, replacing scattered emails, spreadsheets, and phone-only support. It gives customers a way to raise and track complaints, agents a structured queue with full context per case, and admins visibility into team performance — all with role-based access.
- . **Features:**
 - . Role-based registration and login (Customer, Agent, Admin)
 - . Complaint submission, status tracking, agent assignment, and escalation
 - . Real-time-style chat per complaint (polling-based), automatically disabled once a complaint is Closed
 - . Post-resolution feedback and ratings, visible to agents/admins on closed complaints
 - . Agent view of a customer's full past complaint history
 - . Customer profile management (view/edit name, phone, address)
 - . In-app notifications on complaint status changes
 - . Admin feedback analytics dashboard (average rating, rating distribution, agent leaderboard)
 - . CORS-restricted API access and JWT-based route protection

3. Architecture

- . **Frontend:** Built with React (Vite). Pages are organized by role-driven views (Dashboard, ComplaintDetail, Profile, Agents, AgentProfile, FeedbackAnalytics, Notifications), with AuthContext managing the logged-in user's state and token via localStorage. Axios (api/axios.jsx) centralizes API calls, using VITE_API_URL to target the backend. ProtectedRoute gates access by login state and role.
- . **Backend:** Built with Node.js and Express.js in an MVC pattern. Routes (authRoutes, complaintRoutes, feedbackRoutes, messageRoutes, notificationRoutes) delegate to controllers, which contain business logic and talk to Mongoose models. Middleware (auth, adminOnly, agentOrAdmin) enforces JWT verification and role-based access on protected routes. CORS is restricted to the deployed frontend origin only.
- . **Database:** MongoDB, accessed via Mongoose. Core collections: User (fullName, email, password hash, phone, address, role), Complaint (title, description, status, customer, assignedAgent, escalation fields), Message (complaintId, sender,

content), Feedback (complaintId, customer, agent, rating, comments), Notification (user, message, read status). Hosted on MongoDB Atlas in production.

4. Setup Instructions

Prerequisites:

- . Node.js (v18+ recommended)
- . npm
- . A MongoDB connection string (local MongoDB or MongoDB Atlas)

Installation:

- . 1. Clone the repository: git clone <https://github.com/saitarun1806/CUSTOMER-REGISTRY.git>
- . 2. Backend setup:
 - . cd server
 - . npm install
 - . Create a .env file in server/ with:
 - . PORT=8081
 - . MONGO_URI=<your MongoDB connection string>
 - . JWT_SECRET=<your JWT secret>
 - . JWT_EXPIRE=30d
 - . ADMIN_EMAIL=<admin login email>
 - . ADMIN_PASSWORD=<admin login password>
- . 3. Frontend setup:
 - . cd client
 - . npm install
 - . Create a .env file in client/ with:
 - . VITE_API_URL=<your backend URL, e.g. http://localhost:8081>

5. Folder Structure

Client: client/

- . └─ src/
 - . └─ api/axios.jsx — Axios instance, reads VITE_API_URL
 - . └─ components/ — Navbar, ProtectedRoute, ComplaintCard
 - . └─ context/AuthContext.jsx — login state, token, updateUser()
 - . └─ pages/ — Home, Login, Register, Dashboard, ComplaintDetail, Profile, Agents, AgentProfile, FeedbackAnalytics, Notifications
 - . └─ App.jsx — route definitions
 - . └─ main.jsx — app entry point

Server: server/

- . └─ server.js — entry point, connects DB, starts Express app
- . └─ src/
 - . └─ config/db.js — MongoDB connection
 - . └─ models/ — User, Complaint, Message, Feedback, Notification
 - . └─ controllers/ — business logic per resource
 - . └─ routes/ — Express routers per resource
 - . └─ middleware/authMiddleware.js — JWT auth, adminOnly, agentOrAdmin

6. Running the Application

. Backend:

cd server

npm start

Runs on `http://localhost:8081` (or the PORT set in `.env`).

. Frontend:

cd client

npm run dev

Runs on `http://localhost:5173` via Vite's dev server, calling the backend at `VITE_API_URL`.

7. API Documentation

. Auth (`/v1/api/auth`)

Method: POST, Endpoint: `/register`, Access: Public, Description: Register a new customer

Method: POST, Endpoint: `/login`, Access: Public, Description: Log in (also auto-provisions the admin account on first use)

Method: POST, Endpoint: `/agent`, Access: Admin, Description: Create a new agent account

Method: GET, Endpoint: `/agents`, Access: Admin, Description: List all agents

Method: PUT, Endpoint: `/profile`, Access: Authenticated, Description: Update own `fullName/phone/address`

. Complaints (`/api/complaints`)

Method: POST, Endpoint: `/`, Access: Authenticated, Description: Create a complaint

Method: GET, Endpoint: `/`, Access: Authenticated, Description: List complaints (scoped by role)

Method: PUT, Endpoint: `/:id/assign`, Access: Admin, Description: Assign an agent to a complaint

Method: PUT, Endpoint: `/:id/status`, Access: Agent/Admin, Description: Update complaint status

Method: PUT, Endpoint: `/:id/escalate`, Access: Agent/Admin, Description: Escalate a complaint with a reason

Method: GET, Endpoint: `/status/:status`, Access: Admin, Description: Filter complaints by status

Method: GET, Endpoint: `/customer/:customerId`, Access: Agent/Admin, Description: Get a customer's full complaint history

. Feedback (`/api/feedback`)

Method: POST, Endpoint: `/:complaintId`, Access: Authenticated, Description: Submit feedback for a complaint

Method: GET, Endpoint: `/agent/:agentId`, Access: Agent/Admin, Description: Get all feedback for one agent

Method: GET, Endpoint: `/:complaintId`, Access: Authenticated, Description: Get feedback for a specific complaint

Method: GET, Endpoint: `/Admin`, Access: Admin, Description: Get all feedback (used by Analytics page)

- **Messages (/api/messages)**

Method: POST, Endpoint: /:complaintId, Access: Authenticated, Description: Send a message on a complaint

Method: GET, Endpoint: /:complaintId, Access: Authenticated, Description: Get all messages for a complaint

- **Notifications (/api/notifications)**

Method: GET, Endpoint: /, Access: Authenticated, Description: Get the logged-in user's notifications

Method: PUT, Endpoint: /:id/read, Access: Authenticated, Description: Mark a notification as read

8. Authentication

Authentication uses JWT (JSON Web Tokens). On successful login/registration, the backend signs a token containing the user's ID (JWT_SECRET, expiring per JWT_EXPIRE). The frontend stores the token and user object in localStorage via AuthContext, and Axios attaches the token to the Authorization header on every request. Backend middleware (auth) verifies the token and attaches the user to req.user; adminOnly and agentOrAdmin middleware further restrict specific routes by role. The admin account isn't manually registered — it's auto-created on first login using ADMIN_EMAIL/ADMIN_PASSWORD from the server's environment variables.

9. User Interface

- [Insert real screenshots from your deployed app here — Login, Dashboard (per role), Complaint Detail with chat, Profile page, Feedback Analytics dashboard.]

10. Testing

- Manual User Acceptance Testing was performed covering both positive and negative scenarios — registration, login (including invalid credentials), complaint lifecycle (raise, assign, update status, escalate), chat behavior (including disabled state on Closed complaints), feedback submission and visibility, customer history, profile updates, role-based access restrictions, and CORS enforcement. See the separate User Acceptance Testing (UAT) document for the full list of 20 test cases and results.

11. Screenshots or Demo

- Live Demo: <https://customer-registry-indol.vercel.app>

12. Known Issues

- Chat updates via periodic polling rather than WebSockets, so there can be a brief delay before a new message appears for the other party.
- There is no GET /api/complaints/:id endpoint — the complaint detail page fetches the full complaint list and filters client-side, which is inefficient as complaint volume grows.
- No file/image attachment support in complaints or chat.
- No custom-fields feature — complaint fields are fixed, not admin-configurable.

- . The backend runs on Render's free tier, which spins down after inactivity and adds noticeable delay (~50 seconds) to the first request afterward.
- . No SMS or email notification delivery — notifications are in-app only.

13. Future Enhancements

- . Replace message polling with WebSockets for true real-time chat
- . Add a dedicated GET /api/complaints/:id endpoint to avoid client-side filtering
- . Add file/image attachments to complaints and chat messages
- . Integrate SMS and email notification delivery
- . Add configurable custom fields for complaints